HTTP

Запросы и ответы

От браузера до Java-сервера

GET /tasks

200 OK

Сегодня: от запроса до своего API

- Что происходит между клиентом и сервером.
- Как читать запрос и ответ HTTP/1.1.
- Как выбирать методы, заголовки и статусы.
- Как отправлять запросы через curl.

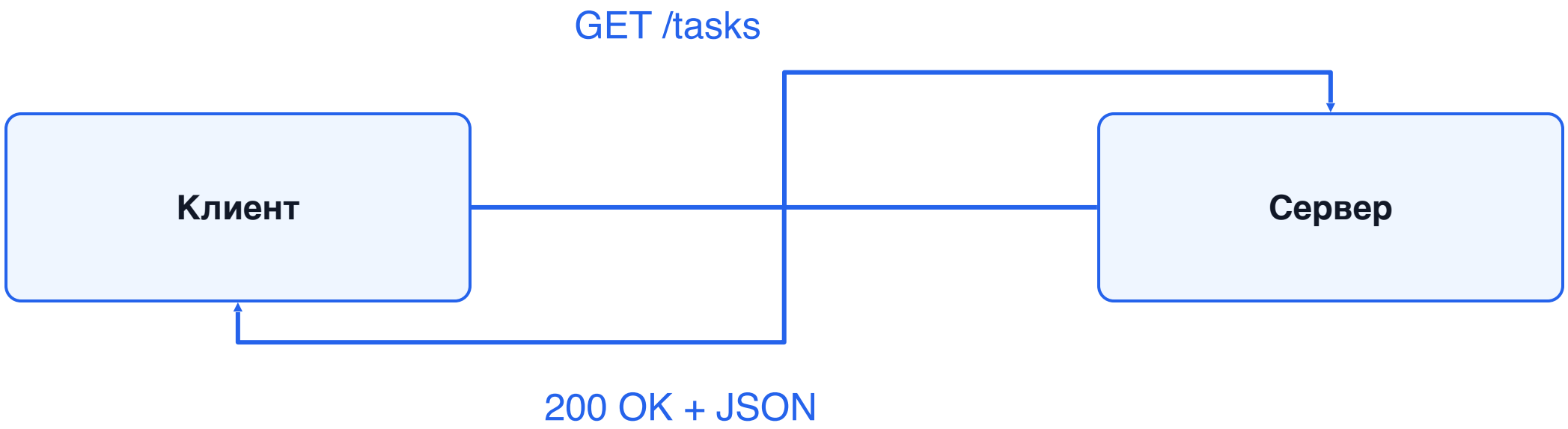
После презентации — запускаем Java-сервер целиком.

Открыли браузер. Ввели адрес. Что дальше?

- Браузер разбирает URL и определяет, куда обращаться.
- При необходимости находит IP-адрес через DNS.
- Устанавливает соединение; для HTTPS — защищённое.
- Отправляет HTTP-запрос и получает ответ.

Одна страница часто порождает много запросов.

Клиент спрашивает — сервер отвечает



Браузер, мобильное приложение и curl могут быть клиентами.

HTTP — договорённость о сообщениях

- Hypertext Transfer Protocol — протокол прикладного уровня.
- Определяет устройство и смысл запросов и ответов.
- Запрос адресован ресурсу: странице, файлу, задаче…
- JSON — формат данных, который можно передать внутри HTTP.

HTTP и JSON — разные уровни одной задачи.

URL: адрес ресурса

Разбираем адрес учебного API

```
http://localhost:8080/tasks?done=false
```

http — схема

localhost — хост

8080 — порт

/tasks — путь

done=false — query

Путь и параметры участвуют в выборе данных.

localhost:8080 — куда мы подключаемся?

- localhost обозначает текущий компьютер.
- Порт помогает попасть в нужный серверный процесс.
- Наш Java-сервер будет слушать порт 8080.
- У HTTP порт по умолчанию 80, у HTTPS — 443.

На телефоне localhost — сам телефон, а не ваш ноутбук.

Что находится под HTTP

Уровень	За что отвечает
HTTP	Смысл запросов и ответов
TLS — для HTTPS	Шифрование и проверка подлинности сервера
TCP — для HTTP/1.1	Упорядоченный поток байтов
IP	Доставка пакетов между адресами

Сегодня читаем HTTP/1.1, а не пишем сетевой стек.

HTTPS: HTTP через защищённое соединение

- Содержимое обмена шифруется при передаче.
- Сертификат помогает проверить сервер.
- Методы, статусы, заголовки и тело никуда не исчезают.
- Шифрование не заменяет проверку прав в приложении.

Для локального примера — HTTP. Для реального сервиса — HTTPS.

HTTP/1.1, HTTP/2, HTTP/3

Версия	Как передаются сообщения
HTTP/1.1	Текстовые стартовая строка и заголовки
HTTP/2	Бинарные кадры; несколько потоков в соединении
HTTP/3	Бинарные кадры; транспорт QUIC

Смысл методов и статусов сохраняется между версиями.

Запрос: просим список задач

Пример HTTP/1.1

```
GET /tasks HTTP/1.1
Host: localhost:8080
Accept: application/json
```

GET — метод.

/tasks — цель запроса.

HTTP/1.1 — версия.

Далее заголовки и пустая строка.

У этого запроса тела нет.

Запрос с телом: создаём задачу

Пример HTTP/1.1

```
POST /tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Content-Length: 15

{"title":"Git"}
```

До пустой строки — метаданные.

После неё — тело запроса.

15 — число байтов тела, не строк.

Тело передаёт данные, с которыми работает сервер.

Ответ: задача создана

Пример HTTP/1.1

```
HTTP/1.1 201 Created
Content-Type: application/json
Content-Length: 35
Location: /tasks/1

{"id":1,"title":"Git","done":false}
```

Первая строка — версия и статус.

Location — адрес созданной задачи.

Тело — представление результата.

Запрос и ответ — два разных сообщения.

Где заканчиваются заголовки и тело?

- Пустая строка отделяет заголовки от тела.
- Content-Length задаёт длину тела в байтах.
- В HTTP/1.1 есть и другие правила определения длины.
- Например, chunked передаёт тело порциями.

Разбор сообщений поручаем HTTP-библиотеке.

Основные методы

Метод	Смысл	Наш пример
GET	Получить представление	GET /tasks
POST	Обработать данные; часто создать	POST /tasks
PUT	Создать или полностью заменить	PUT /tasks/1
PATCH	Частично изменить	PATCH /tasks/1
DELETE	Удалить связь ресурса с URI	DELETE /tasks/1

Метод задаёт намерение, а не имя Java-метода.

Ещё два полезных метода

Метод	Что делает	Когда полезен
HEAD	Как GET, но без тела ответа	Проверить метаданные
OPTIONS	Запрашивает варианты взаимодействия	Узнать возможности ресурса

HEAD — отдельный метод, а не «GET с обрезанным выводом».

Повторить запрос — всегда безопасно?

- GET и HEAD не должны запрашивать изменение состояния.
- PUT и DELETE по смыслу идемпотентны.
- Повтор одинакового запроса даёт тот же ожидаемый эффект.
- Повтор POST может создать вторую задачу.

Идемпотентность — про эффект, а не одинаковый ответ.

Путь, query и тело: куда положить данные?

Часть	Пример	Роль в нашем API
Путь	/tasks/42	Какой ресурс
Query	/tasks?done=false	Как отобрать задачи
Тело	{"title":"Git"}	Данные для создания

Соглашение API должно быть понятным и последовательным.

Заголовки: метаданные сообщения

Заголовок	Что сообщает
Host	К какому хосту обращён запрос HTTP/1.1
Content-Type	Формат тела этого сообщения
Accept	Какие форматы ответа подходят клиенту
Content-Length	Сколько байтов занимает тело
Authorization	Данные аутентификации для запроса

Имена заголовков регистронезависимы.

Content-Type и Accept — не одно и то же

Пример HTTP/1.1

```
POST /tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Accept: application/json
Content-Length: 15

{"title":"Git"}
```

Content-Type:
«Я отправляю JSON».

Accept:
«Мне подходит ответ в JSON».

Формат запроса и формат ответа выбираются отдельно.

Тело — это не обязательно JSON

- HTML — документ для браузера.
- JSON — удобный формат структурированных данных.
- Текст, изображение, архив — тоже возможные тела.
- Некоторые запросы и ответы обходятся без тела.

Content-Type описывает формат. Байты — это само содержимое.

Статус: чем закончилась обработка

Группа	Смысл	Пример
1xx	Промежуточная информация	100 Continue
2xx	Успех	200 OK
3xx	Перенаправление / работа с кэшем	304 Not Modified
4xx	Запрос не может быть выполнен	404 Not Found
5xx	Ошибка на стороне сервера	500 Internal Server Error

Статус — машинно читаемый результат, а не украшение ответа.

Успешные ответы: 200, 201, 204

Статус	Когда используем	Тело
200 OK	Получили список задач	JSON со списком
201 Created	Создали задачу	Можно вернуть задачу
204 No Content	Успешно, содержимого нет	Отсутствует

Пустой список — обычно 200 и [], а не 404.

Ошибки запроса: что различать

Статус	Смысл в примере
400 Bad Request	Некорректный JSON или данные запроса
401 Unauthorized	Нет подходящих данных аутентификации
403 Forbidden	Сервер отказывается разрешить действие
404 Not Found	Ресурс не найден
405 Method Not Allowed	Путь есть, но метод не поддерживается

При 405 сервер сообщает допустимые методы в Allow.

500 — не то же самое, что «нет связи»

- 500: сервер прислал HTTP-ответ о своей ошибке.
- Connection refused: подключение не удалось установить.
- Timeout: за отведённое время не дождались результата.
- Ошибка DNS: не удалось определить адрес сервера.

Если HTTP-ответа нет, HTTP-статуса тоже нет.

HTTP не помнит диалог сам по себе

- Каждый запрос содержит сведения, нужные для его обработки.
- Приложение может хранить задачи, пользователей и сессии.
- Cookie или токен помогают связать запрос с пользователем.
- Соединение TCP и пользовательская сессия — разные вещи.

Stateless-протокол не запрещает серверу хранить данные.

curl — клиент HTTP в терминале

- Отправляет запросы без интерфейса браузера.
- Позволяет задать URL, метод, заголовки и тело.
- Удобен для проверки API и воспроизведения ошибок.
- Команду можно сохранить и повторить.

curl показывает, что API доступно не только браузеру.

Первый запрос через curl

Терминал • команда и пример ответа

```
curl --http1.1 -i http://localhost:8080/tasks

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 2

[]
```

По умолчанию — GET.

-i добавляет заголовки ответа к выводу.

[] — пустой список.

curl сам формирует сообщение HTTP.

Три похожих флага curl

Флаг	Что меняет
-i / --include	Показывает заголовки вместе с телом ответа
-I / --head	Отправляет HEAD; тела ответа не будет
-v / --verbose	Показывает подробности обмена и соединения

Маленькая -i и большая -I делают разное.

Создаём задачу: JSON из файла

Bash / zsh • в других оболочках команда одной строкой

```
curl --http1.1 -i \
  -H "Content-Type: application/json" \
  --data-binary @task.json \
  http://localhost:8080/tasks

Содержимое task.json:
{"title":"Git"}
```

-H добавляет заголовок.

@task.json читает файл.

--data-binary отправляет байты тела и выбирает POST.

Файл избавляет от сложного экранирования JSON в терминале.

Получаем ответ и проверяем результат

Пример ответа на POST /tasks

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /tasks/1
Content-Length: 35

{"id":1,"title":"Git","done":false}

Затем повторяем GET /tasks
```

201 — создание успешно.

Location — новый ресурс.

Следующий GET должен вернуть список с задачей.

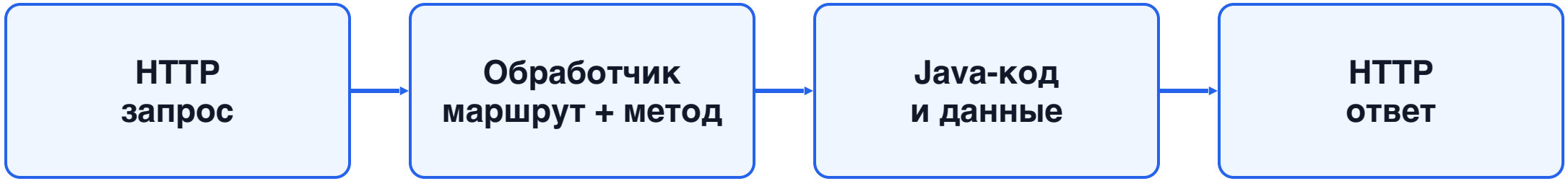
Проверяем и статус, и содержимое ответа.

Договоримся о контракте нашего API

Запрос	Результат	Успех
GET /hello	Текст Hello, HTTP!	200
GET /tasks	JSON-массив задач	200
POST /tasks	JSON: title → созданная задача	201

Контракт — что клиент отправляет и что получает.

Как запрос проходит через Java-сервер



Jackson связывает JSON в сообщениях и объекты Java.

Что берём для первого сервера

Компонент	Его задача
JDK HttpServer	Принять HTTP и вызвать обработчик
Наш Java-код	Проверить запрос, создать или найти данные
Jackson	Прочитать и записать JSON
Maven	Подключить библиотеку и собрать проект

HTTP не требует Spring, чтобы начать с ним работать.

Три ошибки, которых уже можно избежать

- Не отвечать 200 на любую ошибку.
- Не путать Content-Type своего тела и Accept клиента.
- Не считать любой ответ сервера JSON: сначала смотрим статус и заголовки.

Сначала контракт и смысл. Потом код обработчика.

Где смотреть точные правила

- RFC 9110 — семантика HTTP: методы, статусы, поля.
- RFC 9112 — устройство сообщений HTTP/1.1.
- MDN HTTP — справочник с примерами.
- curl manual и JDK HttpServer API — инструменты практики.

Ссылки на документы — в заметках к слайдам.

Теперь собираем сервер

Запускаем → отправляем curl → читаем ответ
Создаём задачу → получаем список → проверяем ошибки